

Price Impact HTE Analysis Pipeline

Code Review & Error Analysis Report

Brian Butler

June 3, 2026

Project: Market/
Language: Python 3.13
Libraries: econml, scikit-learn, DoWhy, pandas, NumPy
Files: 8 source files, 718 total lines

Contents

1	Project Overview	3
1.1	Data Flow	3
1.2	File Index	3
2	Configuration — <code>config/params.yaml</code>	4
2.1	Error Analysis	5
3	Data Generation — <code>src/dgp.py</code>	6
3.1	Purpose	6
3.2	Key Functions	6
3.2.1	<code>load_config()</code>	6
3.2.2	<code>compute_true_cate()</code>	6
3.2.3	<code>generate_data()</code>	6
3.2.4	<code>split_data()</code>	7
3.3	Error Analysis	7
4	Feature Engineering — <code>src/features.py</code>	8
4.1	Purpose	8
4.2	Key Functions	8
4.2.1	<code>winsorize()</code>	8
4.2.2	<code>add_liquidity_features()</code>	8
4.2.3	<code>standardize()</code>	8
4.2.4	<code>engineer_features()</code> — Full Pipeline	8
4.3	Error Analysis	9
5	Causal Graph — <code>src/causal_graph.py</code>	10
5.1	Purpose	10
5.2	Causal DAG	10
5.3	Identifying Assumptions	10
5.4	Error Analysis	10
6	Estimators — <code>src/estimators.py</code>	11
6.1	Purpose	11
6.2	Treatment Binarisation	11
6.3	EstimatorSuite Class	11
6.4	Error Analysis	12
7	Evaluation — <code>src/evaluation.py</code>	13
7.1	Purpose	13
7.2	Metrics	13
7.3	Key Function: <code>evaluate_all()</code>	13
7.4	Error Analysis	14

8	Visualization — <code>src/visualization.py</code>	15
8.1	Purpose	15
8.2	Plot Functions	15
8.3	Error Analysis	15
9	Pipeline Orchestration — <code>main.py</code>	16
9.1	Pipeline Steps	16
9.2	Error Analysis	16
10	Cross-Cutting Issues & Summary	17
10.1	Issue Severity Summary	17
10.2	All High-Severity Issues	17
10.3	All Medium-Severity Issues	17
10.4	Recommended Fix Order	18

1 Project Overview

This project implements a **Heterogeneous Treatment Effect (HTE)** analysis pipeline for measuring **price impact** in financial markets. It:

1. Generates synthetic trade data with a known ground-truth CATE,
2. Engineers liquidity and volatility features,
3. Fits five causal ML estimators (CausalForest + four meta-learners),
4. Evaluates estimator accuracy against the known ground truth, and
5. Produces publication-quality visualizations.

The ground-truth Conditional Average Treatment Effect is defined analytically as:

$$\tau(\mathbf{X}) = \beta_0 + \frac{\beta_1}{\text{depth}} + \beta_2 \cdot \text{volatility} \quad (1)$$

where $\beta_0 = 0.0005$, $\beta_1 = 0.5$, and $\beta_2 = 0.05$.

1.1 Data Flow

The pipeline executes six sequential steps, orchestrated by `main.py`:

Step	Module	Action	Output
1	<code>dgp.py</code>	Generate synthetic data	DataFrame (200k rows)
2	<code>features.py</code>	Engineer features	Standardised covariates
3	<code>estimators.py</code>	Fit 5 HTE estimators	Trained model objects
4	<code>estimators.py</code>	Predict CATE on test set	CATE arrays + CIs
5	<code>evaluation.py</code>	Compute accuracy metrics	Results DataFrame
6	<code>visualization.py</code>	Generate plots	PNG files in <code>results/</code>

1.2 File Index

File	Path	Lines	Role
<code>params.yaml</code>	<code>config/</code>	66	All hyperparameters
<code>dgp.py</code>	<code>src/</code>	98	Synthetic data generation
<code>features.py</code>	<code>src/</code>	86	Feature engineering
<code>causal_graph.py</code>	<code>src/</code>	69	Causal DAG & assumptions
<code>estimators.py</code>	<code>src/</code>	129	Five HTE estimators
<code>evaluation.py</code>	<code>src/</code>	127	Metrics & GATES analysis
<code>visualization.py</code>	<code>src/</code>	193	All plotting functions
<code>main.py</code>	<code>root</code>	85	Pipeline orchestration

2 Configuration — config/params.yaml

All hyperparameters are centralised in a single YAML file. The configuration is loaded by `dgp.load_config()` and threaded through every module.

```

1  random_seed: 42
2
3  dgp:
4    n_observations: 200000
5    train_fraction: 0.7
6
7    # Covariate distributions
8    spread_loc: -2.0
9    spread_scale: 0.6
10   depth_scale: 50.0
11   volatility_scale: 0.02
12
13   # Treatment (confounded by market state)
14   treatment_noise_scale: 5.0
15   treatment_depth_coef: 0.3
16   treatment_vol_coef: -10.0
17
18   # Outcome -- true CATE: tau(X) = B0 + B1/depth + B2*vol
19   baseline_drift_scale: 0.001
20   cate_intercept: 0.0005
21   cate_depth_coef: 0.5
22   cate_vol_coef: 0.05
23   outcome_noise_scale: 0.002
24
25   # AR(1) autocorrelation in noise (0 = IID)
26   ar1_coefficient: 0.3
27
28  features:
29    volatility_window: 20
30    winsorize_percentile: 0.01
31
32  estimators:
33    causal_forest:
34      n_estimators: 1000
35      min_samples_leaf: 20
36      max_depth: null
37      discrete_treatment: false
38
39    s_learner:
40      base_model: "GradientBoostingRegressor"
41      n_estimators: 200
42
43    t_learner:
44      base_model: "GradientBoostingRegressor"
45      n_estimators: 200
46
47    x_learner:
48      base_model: "GradientBoostingRegressor"
49      n_estimators: 200
50
51    dr_learner:
52      base_model: "GradientBoostingRegressor"
53      n_estimators: 200
54

```

```
55     binary_treatment_quantile: 0.75
56
57     evaluation:
58         n_bootstrap: 200
59         ci_level: 0.90
60         n_cate_bins: 5
61
62     output:
63         results_dir: "results"
64         figure_format: "png"
65         figure_dpi: 300
```

Listing 1: Full configuration file.

2.1 Error Analysis

ID	Param	Sev.	Description
C1	n_bootstrap	Low	Declared but never referenced in any code. Dead config.
C2	figure_dpi	Low	Never read; <code>visualization.py</code> hardcodes DPI to 300.
C3	load_config	Med	No schema validation. A config typo produces a cryptic <code>KeyError</code> .

3 Data Generation — src/dgp.py

3.1 Purpose

Generate synthetic trade data where the ground-truth CATE is known analytically (Equation 1), allowing precise evaluation of estimator accuracy.

3.2 Key Functions

3.2.1 load_config()

Reads the YAML configuration file and returns a nested dictionary.

```
1 def load_config(config_path: str = "config/params.yaml") -> dict:
2     """Load the YAML configuration file."""
3     with open(config_path, "r") as f:
4         return yaml.safe_load(f)
```

Listing 2: Configuration loader.

3.2.2 compute_true_cate()

Implements Equation 1 with a small epsilon guard against division by zero.

```
1 def compute_true_cate(depth, volatility, cfg) -> np.ndarray:
2     dgp = cfg["dgp"]
3     cate = (
4         dgp["cate_intercept"]
5         + dgp["cate_depth_coef"] / (depth + 1e-6)
6         + dgp["cate_vol_coef"] * volatility
7     )
8     return cate
```

Listing 3: Ground-truth CATE computation.

3.2.3 generate_data()

Creates $n = 200,000$ observations with:

- **Covariates:** spread $\sim \text{LogNormal}$, depth $\sim \text{Exp}(50)$, volatility $\sim |\mathcal{N}(0, 0.02)|$
- **Treatment:** trade_size = $0.3 \cdot \text{depth} - 10 \cdot \text{volatility} + \varepsilon$ (confounded)
- **Outcome:** $Y = g(\mathbf{X}) + \tau(\mathbf{X}) \cdot T + \varepsilon_{\text{AR}(1)}$

```
1 # Market-state covariates
2 spread = rng.lognormal(mean=spread_loc, sigma=spread_scale, size=n)
3 depth = rng.exponential(scale=depth_scale, size=n)
4 volatility = np.abs(rng.normal(0, volatility_scale, size=n))
5
6 # Treatment -- confounded by market state
7 trade_size = np.maximum(depth_coef * depth + vol_coef * vol + noise, 1.0)
8
9 # Outcome with optional AR(1) autocorrelation in noise
10 price_change = baseline_drift + true_cate * trade_size + noise
```

Listing 4: Core data generation logic (abbreviated).

3.2.4 split_data()

Performs a temporal (positional) train/test split at 70/30.

```

1 def split_data(df, cfg):
2     frac = cfg["dgp"]["train_fraction"]
3     split_idx = int(len(df) * frac)
4     return df.iloc[:split_idx].copy(), df.iloc[split_idx:].copy()

```

Listing 5: Train/test split.

3.3 Error Analysis

ID	Lines	Sev.	Description
D1	79–83	Med	<code>split_data</code> is called “temporal” but <code>generate_data</code> produces rows with no time index. The split relies on row order never being shuffled—a fragile assumption.
D2	21	Low	<code>depth</code> drawn from <code>Exp(50)</code> can be near zero. The <code>1e-6</code> guard works, but very small depths produce enormous CATEs that may dominate evaluation metrics.
D3	(eval 96)	Med	<code>cfg.copy()</code> is shallow—nested dicts like <code>cfg["dgp"]</code> still reference the original. If any downstream code mutated <code>cfg_copy["dgp"]</code> , it would corrupt the original config. Currently safe but latent.
D4	—	Low	No <code>__init__.py</code> in <code>src/</code> . The <code>sys.path</code> hack in <code>main.py</code> works around this, but relative imports (<code>from .dgp</code>) used in <code>evaluation.py</code> will fail.

4 Feature Engineering — `src/features.py`

4.1 Purpose

Transform raw covariates into modelling features: winsorise extremes, derive liquidity ratios, add rolling volatility, and Z-score standardise.

4.2 Key Functions

4.2.1 `winsorize()`

```
1 def winsorize(series, pct=0.01):
2     lower = series.quantile(pct)
3     upper = series.quantile(1 - pct)
4     return series.clip(lower, upper)
```

Listing 6: Winsorisation: clip extreme values at tail percentiles.

4.2.2 `add_liquidity_features()`

```
1 def add_liquidity_features(df):
2     df = df.copy()
3     df["log_spread"] = np.log1p(df["spread"])
4     df["inv_depth"] = 1.0 / (df["depth"] + 1e-6)
5     df["spread_depth_ratio"] = df["spread"] / (df["depth"] + 1e-6)
6     return df
```

Listing 7: Derived liquidity proxies.

4.2.3 `standardize()`

Z-score normalisation that can reuse training statistics for the test set.

```
1 def standardize(df, columns, fit_stats=None):
2     df = df.copy()
3     if fit_stats is None:
4         fit_stats = {}
5         for col in columns:
6             fit_stats[col] = {"mean": df[col].mean(),
7                               "std": df[col].std()}
8     for col in columns:
9         mu = fit_stats[col]["mean"]
10        sigma = fit_stats[col]["std"]
11        df[col] = (df[col] - mu) / (sigma + 1e-10)
12    return df, fit_stats
```

Listing 8: Z-score standardisation with fit/transform pattern.

4.2.4 `engineer_features()` — Full Pipeline

```
1 def engineer_features(df, cfg, fit_stats=None):
2     pct = cfg["features"]["winsorize_percentile"]
3     for col in ["spread", "depth", "volatility", "trade_size"]:
```

```

4     df[col] = winsorize(df[col], pct)           # BUG: mutates input
5     df = add_liquidity_features(df)
6     df = add_rolling_volatility(df, window=...)
7     df, fit_stats = standardize(df, covariate_cols, fit_stats)
8     return df, fit_stats

```

Listing 9: Master feature engineering function (abbreviated).

4.3 Error Analysis

ID	Lines	Sev.	Description
F1	59–60	High	Mutates input DataFrame. <code>engineer_features</code> applies <code>winsorize</code> to columns <i>before</i> making a copy. The caller’s original DataFrame (<code>train_raw</code> , <code>test_raw</code>) is silently modified in-place.
F2	18–23	High	Rolling volatility uses the outcome variable (<code>price_change</code>). In a real setting this would be data leakage. In this synthetic setup the feature is computed but not used by the estimators, so it wastes computation and creates confusion.
F3	65–68	Med	Feature mismatch. Seven columns are standardised, but <code>main.py</code> only feeds three (<code>spread</code> , <code>depth</code> , <code>volatility</code>) to the estimators. The extra four features are never used.
F4	59–60	Med	Test-set quantile leakage. <code>winsorize</code> recalculates quantile bounds from the test data instead of reusing training bounds. Inconsistent with the <code>standardize</code> function which correctly reuses <code>fit_stats</code> .

5 Causal Graph — `src/causal_graph.py`

5.1 Purpose

Encodes the assumed causal DAG and the four identifying assumptions. Provides a DoWhy `CausalModel` builder for refutation tests.

5.2 Causal DAG

The assumed causal structure is:

```

spread      → trade_size
depth       → trade_size
volatility   → trade_size

spread      → price_change
depth       → price_change
volatility   → price_change
trade_size  → price_change

```

All three market-state variables are **common causes** (confounders) of both treatment and outcome.

5.3 Identifying Assumptions

1. **Unconfoundedness:** Conditional on (spread, depth, volatility), `trade_size` is as-good-as-random. No unobserved confounders affect both T and Y .
2. **Overlap (Positivity):** For every market state $\mathbf{X}$, there is positive probability of observing any trade size T .
3. **SUTVA:** One trade's outcome does not depend on other trades' treatment assignments.
4. **Consistency:** The observed outcome under treatment $T = t$ equals the potential outcome $Y(t)$.

5.4 Error Analysis

ID	Lines	Sev.	Description
G1	—	Low	Module is never imported in <code>main.py</code> . DoWhy refutation tests are defined but never run. Assumptions are documented but not programmatically verified.
G2	48–50	Low	<code>build_dowhy_model</code> silently returns <code>None</code> if <code>dowhy</code> is not installed. Any caller that doesn't check for <code>None</code> would crash.

6 Estimators — `src/estimators.py`

6.1 Purpose

Wrap five HTE estimators from the `econml` library in a unified `EstimatorSuite` class.

6.2 Treatment Binarisation

The meta-learners require a binary treatment. Continuous `trade_size` is converted at the 75th percentile:

```
1 def binarize_treatment(trade_size, quantile=0.75):
2     threshold = np.quantile(trade_size, quantile)
3     return (trade_size >= threshold).astype(int)
```

Listing 10: Binarise continuous treatment at a quantile threshold.

MEDIUM Severity

CausalForest uses the **continuous** treatment ($\partial Y / \partial T$), while all four meta-learners use a **binarised** treatment ($\mathbb{E}[Y(1) - Y(0)]$). These are *different causal quantities*. Comparing their RMSE against the same ground-truth $\tau(\mathbf{X})$ is scientifically problematic (see issue E2).

6.3 EstimatorSuite Class

Method	Description
<code>fit_all()</code>	Fits all five estimators on training data
<code>_fit_causal_forest()</code>	<code>econml.dml.CausalForestDML</code> — continuous treatment
<code>_fit_s_learner()</code>	<code>econml.metalearners.SLearner</code> — single model
<code>_fit_t_learner()</code>	<code>econml.metalearners.TLearner</code> — separate models per arm
<code>_fit_x_learner()</code>	<code>econml.metalearners.XLearner</code> — cross-fitted imputation
<code>_fit_dr_learner()</code>	<code>econml.metalearners.DRLearner</code> — doubly robust
<code>estimate_cate()</code>	Predicts CATE on new covariates for all estimators
<code>confidence_intervals()</code>	Returns CIs where supported (e.g. CausalForest)

6.4 Error Analysis

ID	Lines	Sev.	Description
E1	102	High	Wrong import path. <code>from econml.metalearners import DRLearner</code> is likely incorrect. In standard <code>econml</code> , <code>DRLearner</code> lives in <code>econml.dr</code> , not <code>econml.metalearners</code> . This will raise <code>ImportError</code> at runtime.
E2	40–49	High	Comparing apples to oranges. <code>CausalForest</code> estimates a marginal effect of continuous treatment, while meta-learners estimate a binary ATE. Evaluating both against the same continuous ground-truth $\tau(\mathbf{X})$ is scientifically invalid.
E3	105	Med	Even if the import were fixed, <code>DRLearner(models=base)</code> may not match the constructor— <code>econml.dr.DRLearner</code> typically requires <code>model_regression</code> and <code>model_propensity</code> separately.
E4	126–127	Med	<code>confidence_intervals</code> catches all exceptions with bare <code>except Exception: pass</code> . Numerical failures are silently swallowed.
E5	21–23	Low	<code>_get_base_model</code> silently returns a default model for unknown names. Config typos go unnoticed.

7 Evaluation — src/evaluation.py

7.1 Purpose

Compute accuracy metrics comparing estimated CATE to the ground-truth CATE.

7.2 Metrics

Metric	Function	What it Measures
RMSE	cate_rmse	$\sqrt{\frac{1}{n} \sum (\hat{\tau}_i - \tau_i)^2}$
MAE	cate_mae	$\frac{1}{n} \sum \hat{\tau}_i - \tau_i $
Bias	cate_bias	$\frac{1}{n} \sum (\hat{\tau}_i - \tau_i)$ (positive = overestimation)
Spearman ρ	rank_correlation	Monotonic rank correlation
Kendall τ	rank_correlation	Concordance-based rank correlation
CI Coverage	ci_coverage	Fraction of true CATEs inside the confidence interval
GATES	gates_analysis	Binned estimated vs. true CATE by quintile

7.3 Key Function: evaluate_all()

```

1 def evaluate_all(cate_dict, true_cate, ci_dict=None, n_bins=5):
2     rows = []
3     for name, est_cate in cate_dict.items():
4         row = {"Estimator": name}
5         row["RMSE"] = cate_rmse(est_cate, true_cate)
6         row["MAE"] = cate_mae(est_cate, true_cate)
7         row["Bias"] = cate_bias(est_cate, true_cate)
8         rank_corr = rank_correlation(est_cate, true_cate)
9         row["Spearman_r"] = rank_corr["spearman_r"]
10        row["Kendall_tau"] = rank_corr["kendall_tau"]
11        if ci_dict and name in ci_dict:
12            lower, upper = ci_dict[name]
13            row["CI_Coverage"] = ci_coverage(true_cate, lower, upper)
14        else:
15            row["CI_Coverage"] = np.nan
16        rows.append(row)
17    return pd.DataFrame(rows).set_index("Estimator")

```

Listing 11: Summary builder: runs all metrics for every estimator.

7.4 Error Analysis

ID	Lines	Sev.	Description
V1	90	High	<code>iid_vs_dependent_comparison</code> uses relative import from <code>.dgp</code> import <code>...</code> , which requires <code>src/</code> to be a proper package (with <code>__init__.py</code>). Will raise <code>ImportError</code> if called.
V2	87–114	Low	<code>iid_vs_dependent_comparison</code> is defined but never invoked in <code>main.py</code> . Dead code.
V3	45	Low	GATES <code>pd.qcut</code> may fail if an estimator predicts a constant CATE (only one unique value).
V4	29–31	Low	<code>spearman_p</code> and <code>kendall_p</code> are computed but never included in the results DataFrame. Statistical significance goes unreported.

8 Visualization — `src/visualization.py`

8.1 Purpose

Generate publication-quality plots. All figures are saved to the `results/` directory.

8.2 Plot Functions

Function	Output File	Description
<code>plot_cate_heatmap</code>	<code>cate_heatmap_true.png</code>	2D heatmap of CATE across depth $\times$ volatility bins
<code>plot_cate_scatter</code>	<code>scatter_<est>.png</code>	Estimated vs. True CATE with 45° reference line
<code>plot_estimator_comparison</code>	<code>estimator_comparison_*.png</code>	Horizontal bar chart comparing estimators
<code>plot_gates</code>	<code>gates_<est>.png</code>	Estimated vs. True CATE by quintile bin
<code>plot_partial_dependence</code>	<code>partial_dependence_*.png</code>	Binned avg. CATE vs. each covariate

8.3 Error Analysis

ID	Lines	Sev.	Description
P1	53	Med	<code>np.random.choice</code> uses the global random state. Scatter plots are non-reproducible across runs.
P2	31–32	Med	Heatmap bins “raw” <code>depth</code> and <code>volatility</code> , but due to bug F1 these columns may already be winsorised. Axis labels are misleading.
P3	19	Low	<code>figure_dpi</code> from config is ignored; style hardcodes <code>savefig.dpi: 300</code> .

9 Pipeline Orchestration — `main.py`

9.1 Pipeline Steps

Step	Lines	Action	Detail
Config	25–26	Load config	Read <code>params.yaml</code>
1/6	28–31	Generate data	200k synthetic observations, temporal split
2/6	33–36	Engineer features	Winsorise, derive, standardise
3/6	38–47	Fit estimators	CausalForest + 4 meta-learners
4/6	49–51	Estimate CATE	Predict CATE and CIs on test set
5/6	53–66	Evaluate	All metrics + GATES analysis
6/6	68–75	Visualise & save	Generate plots, write CSV

9.2 Error Analysis

ID	Lines	Sev.	Description
M1	39–41	Med	Only 3 of 7 engineered features are used as estimator covariates. Four features (<code>log_spread</code> , <code>inv_depth</code> , <code>spread_depth_ratio</code> , <code>rolling_vol</code>) are computed but discarded.
M2	51	Low	CI $\alpha = 1 - 0.90 = 0.10$. This matches econml’s convention— correct .
M3	34–35	High	Consequence of F1: after <code>engineer_features(train_raw)</code> , the <code>train_raw</code> DataFrame is silently mutated. When <code>test_raw</code> is later passed to plotting functions, it too has been corrupted.
M4	—	Low	<code>src/</code> has no <code>__init__.py</code> . The <code>sys.path.insert</code> hack is fragile.

10 Cross-Cutting Issues & Summary

10.1 Issue Severity Summary

	Severity	Count	Action
	HIGH	5	Fix before trusting results
	MEDIUM	8	Fix for correctness and robustness
	LOW	7	Nice to fix, non-critical

10.2 All High-Severity Issues

HIGH Severity

ID	File	Issue
E1	estimators.py:102	DRLearner import path is wrong → ImportError at run-time
E2	estimators.py:40-49	Comparing continuous vs. binary treatment effects against the same ground truth
F1	features.py:59-60	engineer_features mutates the input DataFrame in-place
M3	main.py:34-35	Raw data used in visualisation is silently corrupted (consequence of F1)
V1	evaluation.py:90	Relative import from .dgp will fail without __init__.py

10.3 All Medium-Severity Issues

MEDIUM Severity

ID	File	Issue
F2	features.py:18-23	Rolling volatility computed from outcome variable (leakage)
F3	features.py + main.py	4 engineered features computed but never used
F4	features.py:59-60	Winsorisation uses test-set quantiles instead of training bounds
D1	dgp.py:79-83	“Temporal” split on non-temporal data
D3	dgp.py (via eval)	Shallow cfg.copy() leaves nested dicts shared
E3	estimators.py:105	DRLearner(models=base) may not match constructor API
E4	estimators.py:126-127	CI exceptions silently swallowed
P1	visualization.py:53	Non-reproducible scatter sampling

10.4 Recommended Fix Order

1. **Fix DRLearner import (E1)** — the pipeline will crash without this fix.

```
# Change:
from econml.metalearners import DRLearner
# To:
from econml.dr import DRLearner
```

2. **Fix DataFrame mutation (F1 / M3)** — add `df = df.copy()` at the top of `engineer_features`, before winsorisation.

```
def engineer_features(df, cfg, fit_stats=None):
    df = df.copy()          # <-- add this line
    feat_cfg = cfg["features"]
    pct = feat_cfg["winsorize_percentile"]
    ...
```

3. **Add `src/__init__.py` (V1 / M4)** — create an empty file:

```
touch src/__init__.py
```

4. **Address continuous vs. binary CATE comparison (E2)** — either:
 - Compute a separate binary ground-truth CATE for the meta-learners, *or*
 - Use the same treatment type for all estimators.
5. **Reuse training winsorisation bounds (F4)** — store quantile bounds in `fit_stats` and reuse for the test set (same pattern as `standardize`).
6. **Remove or use extra features (F3)** — either add the four unused columns to the estimator covariates, or remove them from `engineer_features`.